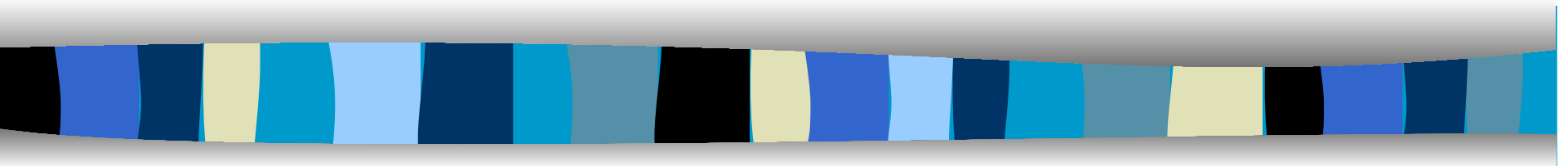


User Authentication



Password, Challenge Response,
Needham Schroeder, Kerberos

User Authentication

- The process of verifying an identity claimed by user or entity for a system entity.
- An entity can be a person, a process, a client, or a server.
- The entity whose identity needs to be proved is called the **claimant**;
- The party that tries to prove the identity of the claimant is called the **verifier**.
- User authentication is the basis for most types of access control and for user accountability.

User Authentication Steps

- **Identification step:** Presenting an identifier to the security system.
- Identifier can be username and its corresponding password.
- These identifiers are registered in the beginning.
- Identification is one to many matching.
- It searches the input identity through registered identities.
- So, it is one to many mapping.

User Authentication Steps

- **Verification step:** Once identity of claimant is found, then input password and registered password is corroborated.
- So, it is one to one mapping.
- If password is matched then, user is authenticated otherwise authentication is denied.

User Authentication types

- User authentication can be based on

-

**something
the individual
knows**

- password,
PIN, answers
to
prearranged
questions

**something
the individual
possesses
(token)**

- smartcard,
electronic
keycard,
physical key

**something
the individual
is (static
biometrics)**

- fingerprint,
retina, face

**something
the individual
does
(dynamic
biometrics)**

- voice pattern,
handwriting,
typing rhythm

User Authentication types

- All these authentication methods have issues.
- Like password can be forgotten.
- Cards can be damaged or lost.
- Finger prints can be damaged.
- Voice can be changed.
- So, we can use combination of these methods.

User Authentication types

- Authentication can be one way or mutual.
- In early days of security system, it was only one way.
- For example users authenticate bank but bank does not authenticate users.
- Email is one way authentication.
- Only sender is authenticated receiver is not authenticated.

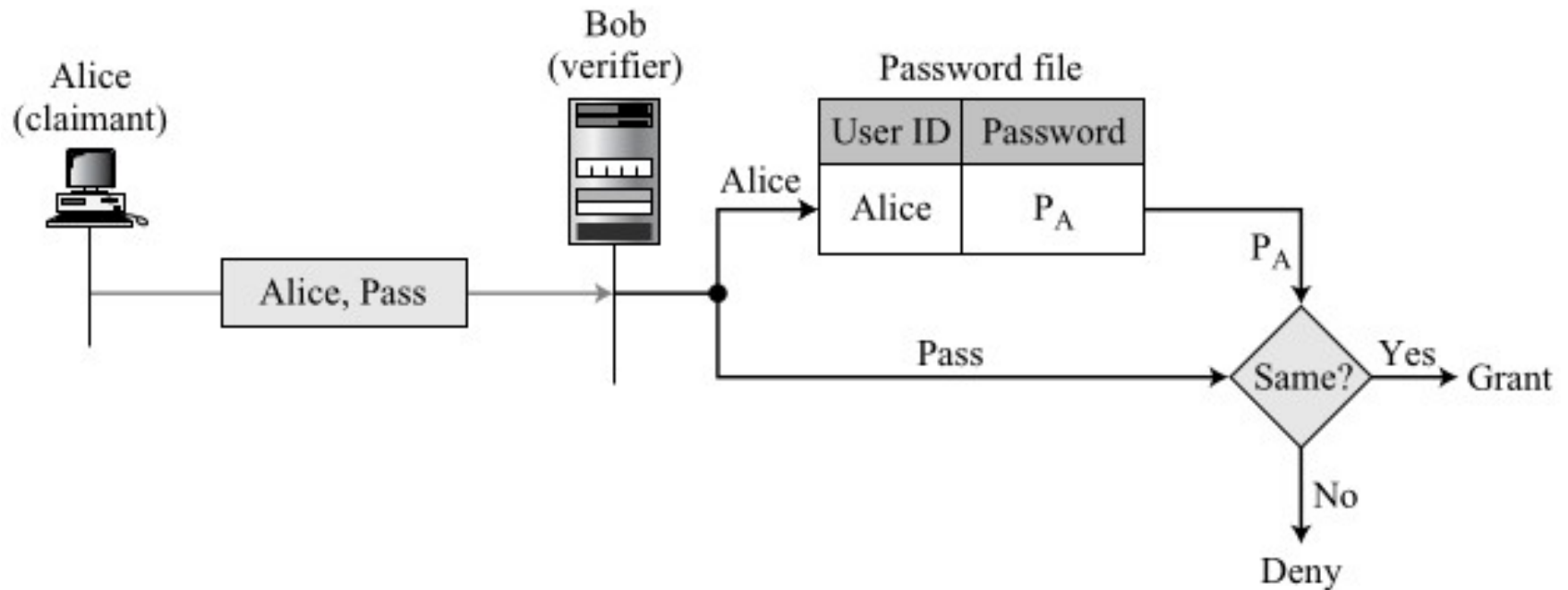
Password Authentication

- The simplest and oldest method of user authentication is the password-based authentication, where the password is something that the claimant knows.
- A password is used when a user needs to access a system to use the system's resources (login).
- Each user has a user identity that is public, and a password that is private.
- One-time password is more secure.

Password Authentication

- The system keeps a table (a file) that is sorted by user identity.
- To access the system resources, the user sends her user identity and password, in plaintext, to the system.
- The system uses the identity to find the password in the table.
- If the password sent by the user matches the password in the table, access is granted; otherwise, it is denied.

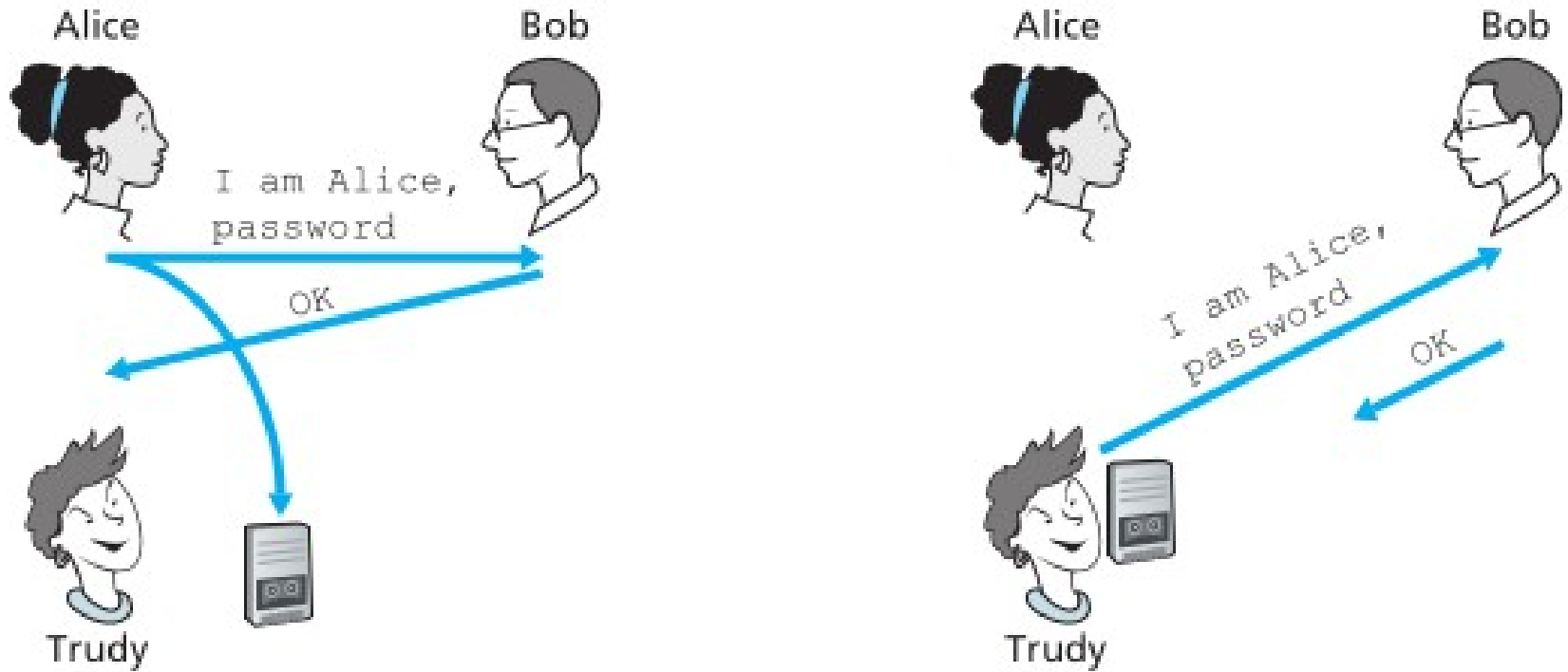
Fixed Password Authentication



attacks on password Authentication

- The simplest **replay attack** is one in which the opponent simply copies a message and replays it later to gain the access to the system.
- Another attack is **dictionary attack**.
- In dictionary attack, attacker has access to password storage file which stores password in hashed form.
- The hacker generates hashed message from all the word in the dictionary.
- And checks if the encrypted password matches with the generated password.

attacks on password Authentication



- Note that encryption of id and password does not help as trudy can get them and replay later.

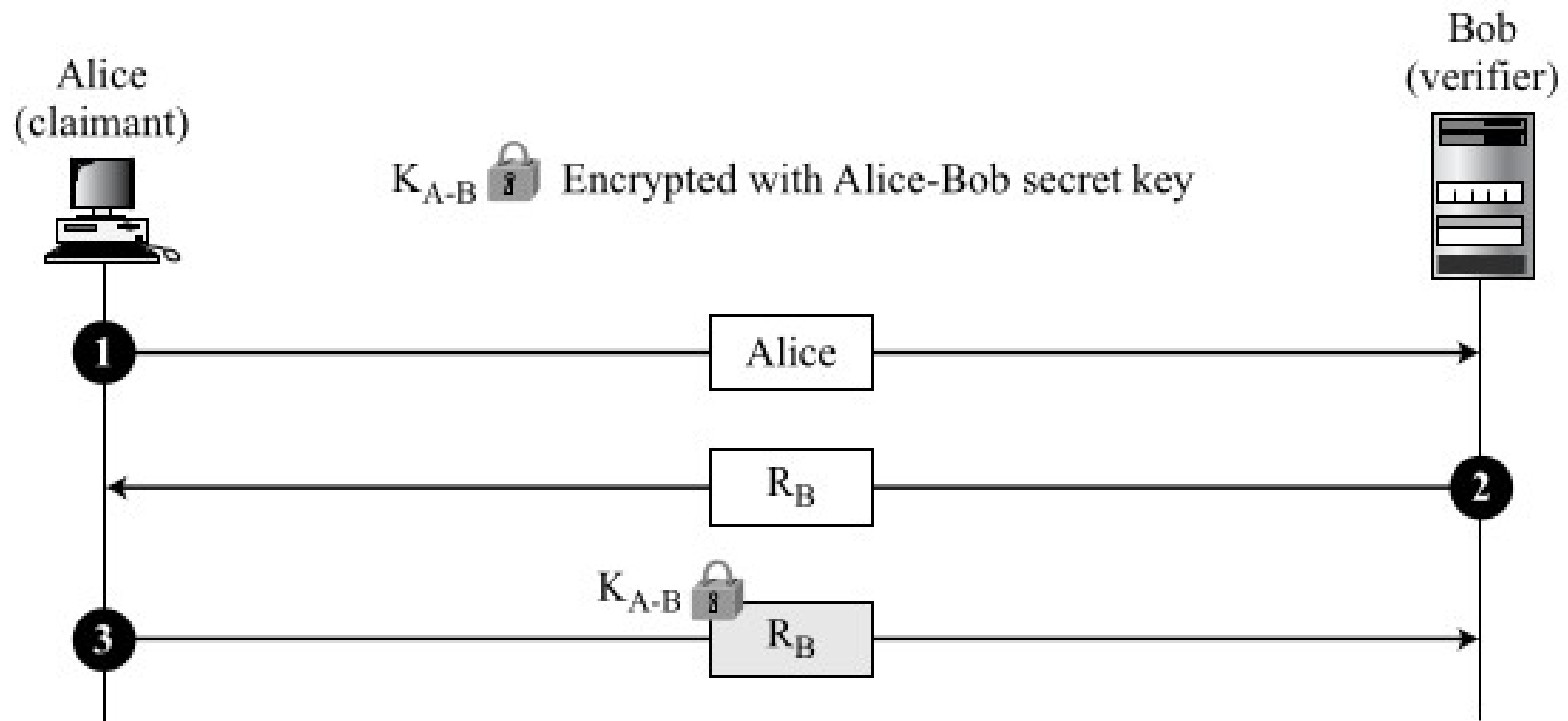
Challenge Response Authentication

- **Symmetric Method**
- We can use nonce to prevent replay attack.
- Nonce is randomly chosen number that is used for only once for authentication.
- It is never repeated by verifier.
- This method is known as challenge response authentication.
- In challenge response method, the user is authenticated without sending the secret key.
- The Alice who wants to be authenticated sends message to verifier that I am Alice and I can prove it

Challenge Response Authentication

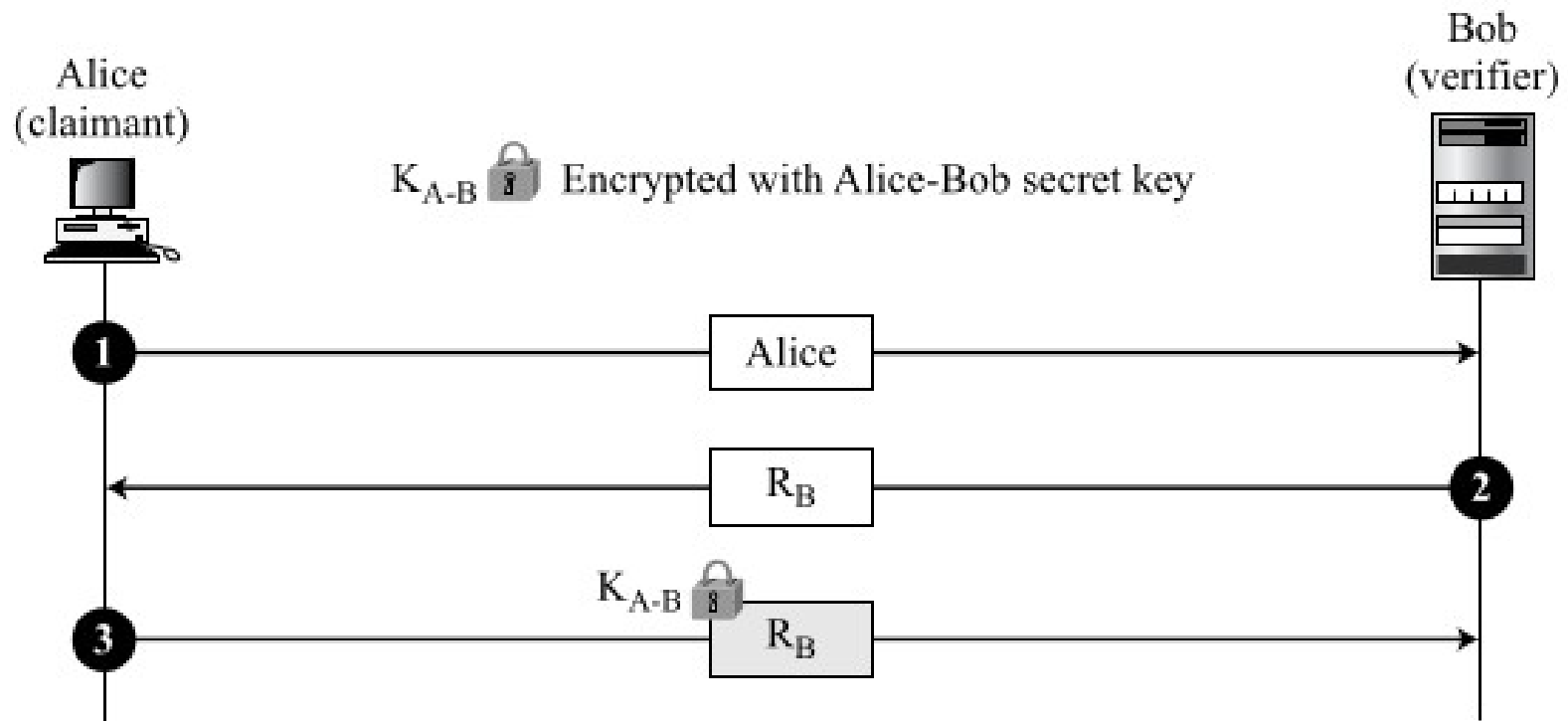
- **Symmetric Method**
- Bob as a Verifier challenges the Alice by sending nonce to prove she has the secret key.
- Alice encrypts the nonce with the secret key shared between Alice and Bob and sends to Bob
- The Bob decrypts the message.
- If decrypted message is same as the nonce sent, then Alice is authenticated otherwise denied.

Challenge Response: Symmetric



- **Symmetric Method**
- Now, recording all these sequence of message does not help Turdy.
- Its because Bob will use fresh nonce for each authentication.

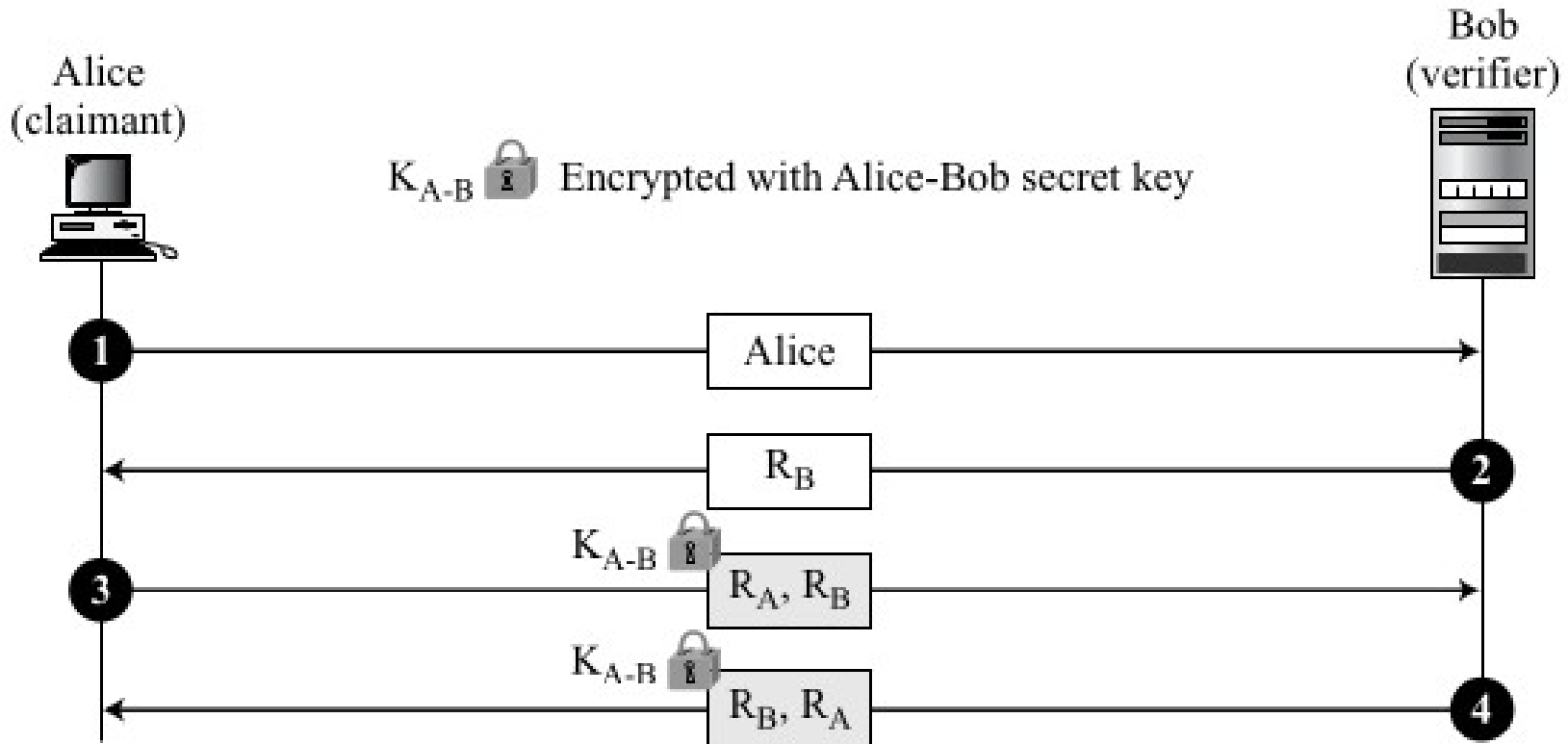
Challenge Response: Symmetric



- Here authentication is only one way.
- In such authentication, Turdy can impersonate as Bob to Alice.
- So, There must be mutual authentication.

Challenge Response: Symmetric

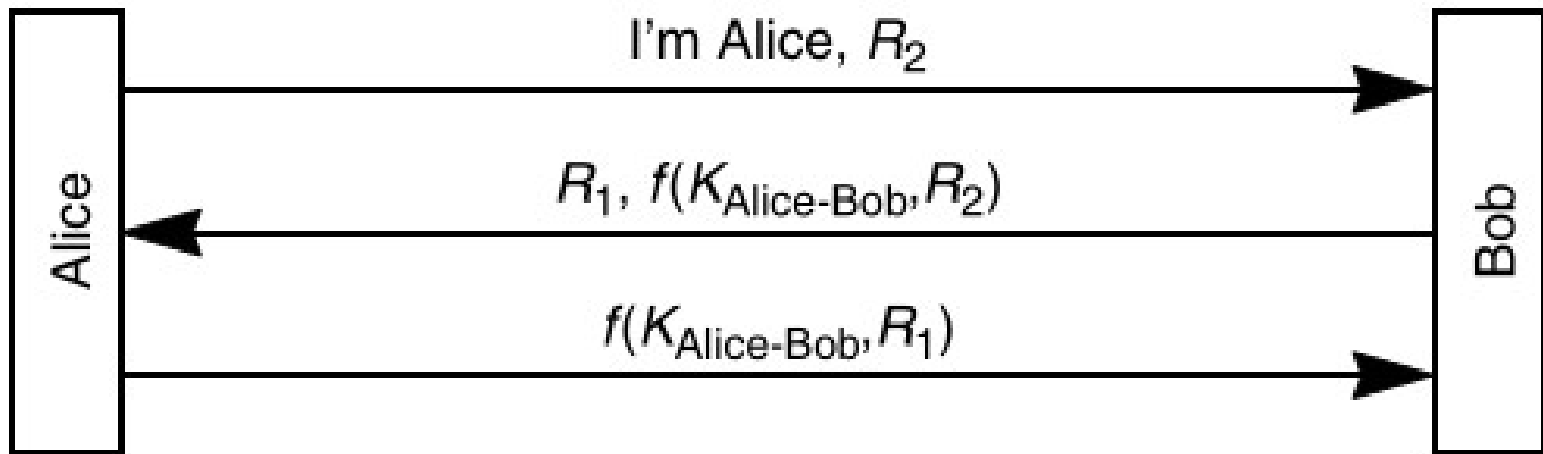
- Mutual Authentication



- This is not efficient as it has four messages.

Challenge Response: Symmetric

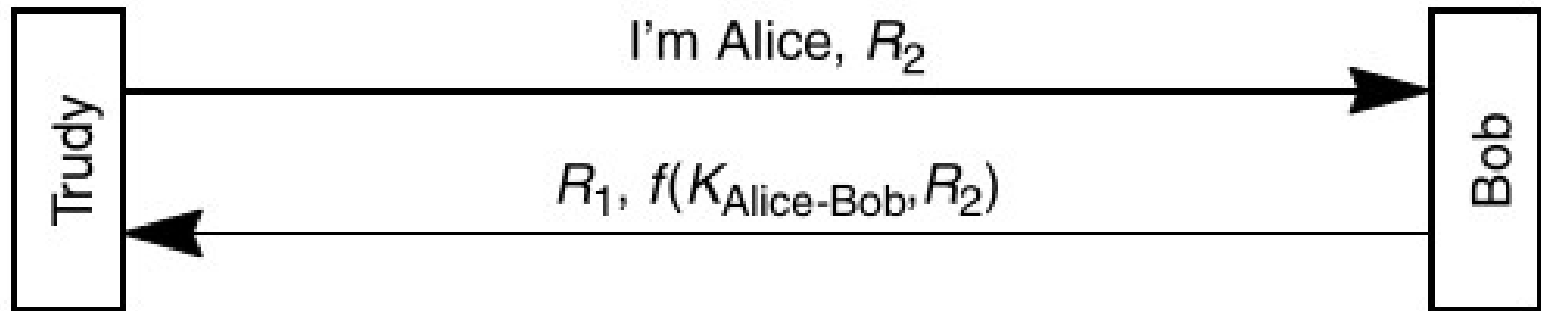
- Mutual Authentication



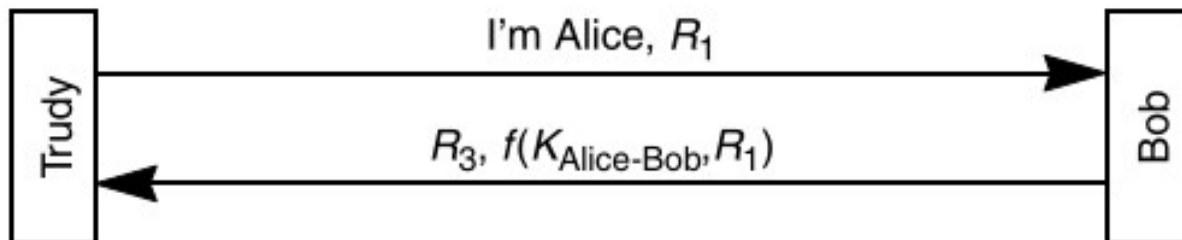
- This mutual authentication can fall into reflection attack.

Challenge Response: Symmetric

- In **reflection attack**, Turdy impersonate as Alice to Bob as given below.

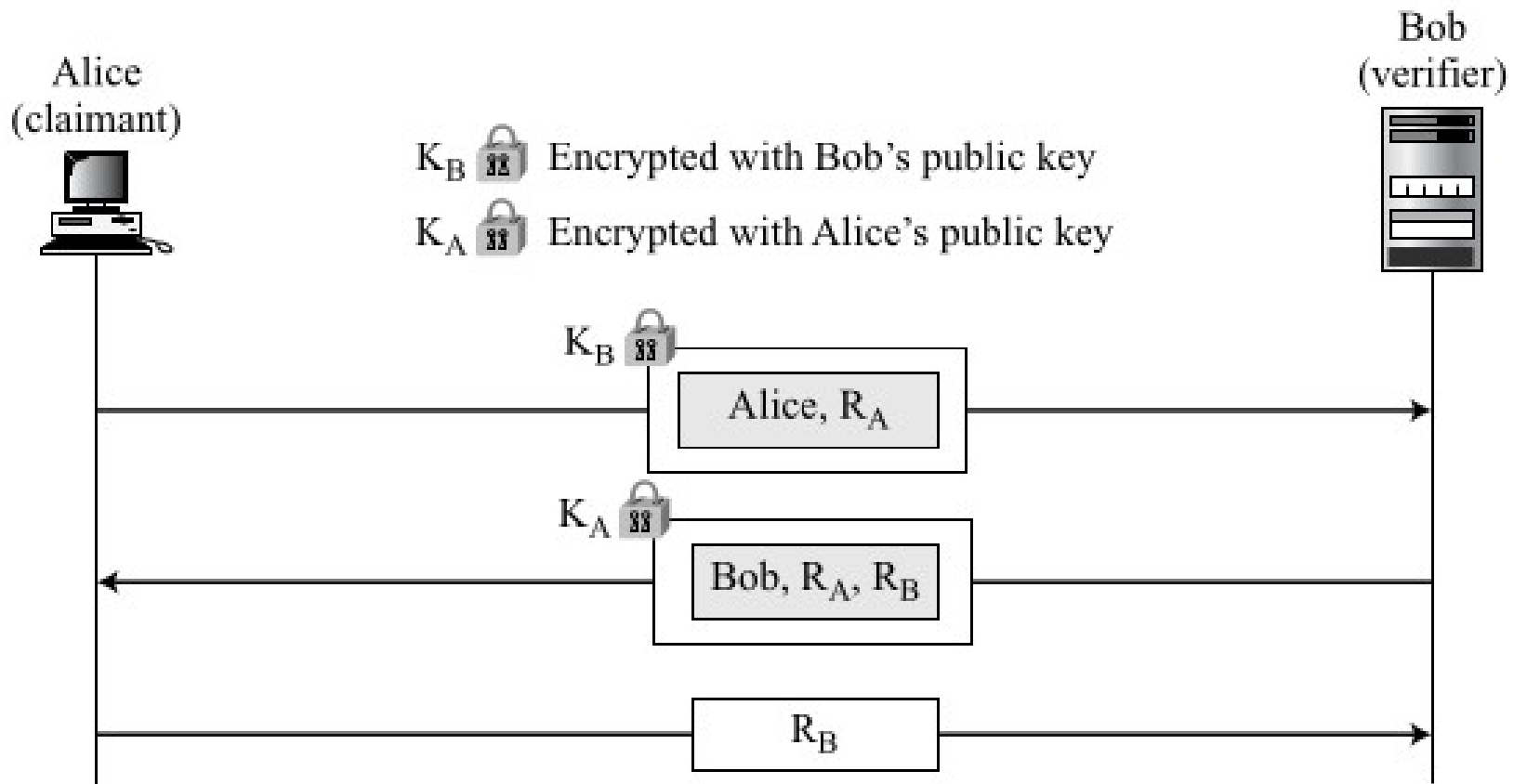


- Now, Turdy cannot encrypt R_1 as it does not own secret key.
- Turdy opens another communication with Bob sending R_1 as its challenge.



Challenge Response: Asymmetric

- Make it efficient with three messages only.

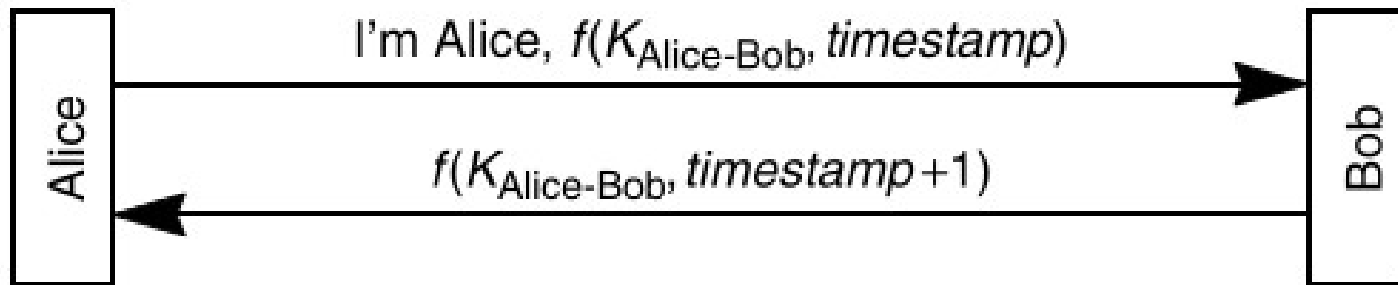


Challenge Response: timestamp

- We can use timestamp to make mutual authentication more efficient.
- Accepts the messages for which the timestamp is within a reasonable tolerance.
- But for this clocks must be synchronized.
- If computer is very fast then message can be replayed.
- So, claimant and verifier must remember each timestamp in communication.

Challenge Response: timestamp

- In this case, we need only two messages.



- Here function f can be public key, symmetric key or use identifier of sender to make the message different.
- We can also add identifier of receiver to make messages different.
- Messages can be replayed if computer is very fast.

Biometric Authentication

- Biometrics is the measurement of physiological or behavioral features that identify a person (authentication by something inherent).
- Biometrics features that cannot be guessed, stolen, or shared.
- Several components are needed for biometrics, including capturing devices, processors, and storage devices.
- **Capturing devices** such as readers (or sensors) measure biometrics features.

Biometric Authentication

- **Processors** change the measured features to the type of data appropriate for saving.
- **Storage devices** save the result of processing for authentication.
- Each person in the community must be **enrolled** in the system by capturing the corresponding feature using capturing devices.
- In **verification**, a person's feature is matched against a single record in the database (one-to-one matching) to find if she is who she is claiming to be.

Biometric Authentication

- In **identification**, a person's feature is matched against all records in the database (one to-many matching) to find if she has a record in the database.
- This is useful, for example, when a company needs to allow access to the building only to employees.

KDC mediated Authentication

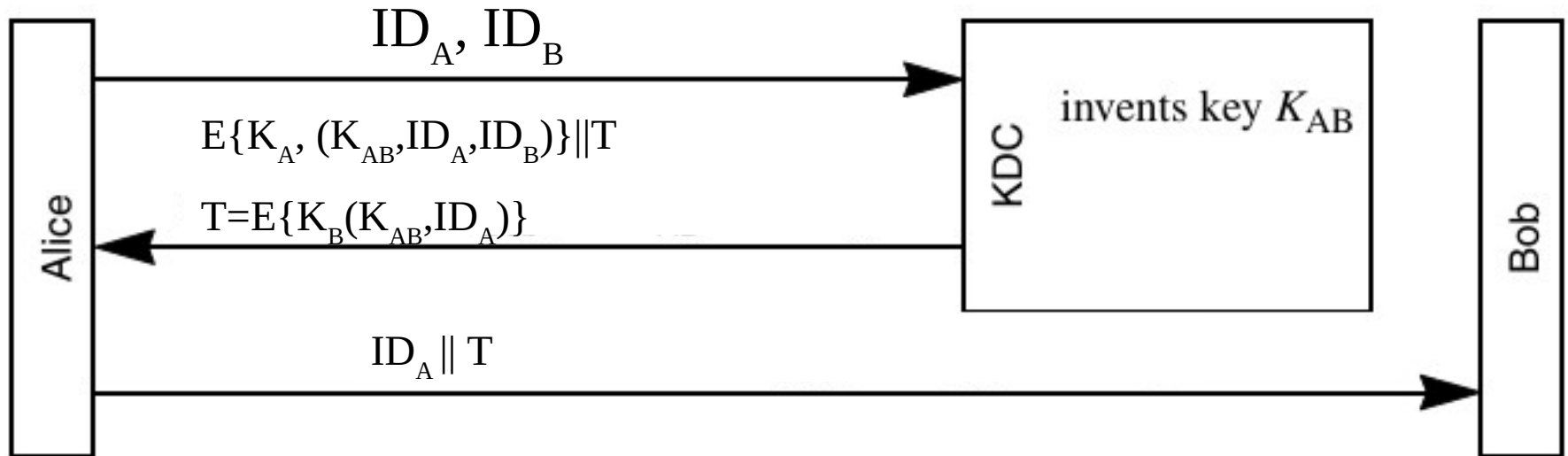
- As we have seen that, for authentication to communicants must agree on shared secret key.
- One method to share secret key is public cryptography.
- But for public key cryptography, we need public key infrastructure.
- For now, we assume that there is no public key cryptography.
- Another method to share secret key is through Key Distribution Center (KDC).

KDC mediated Authentication

- The KDC is responsible for generating keys to be used for a short time over a connection between two parties known as session keys.
- Suppose Alice wants to communicate with Bob.
- Alice and Bob are registered with IDs ID_A , and ID_B .
- Their secret key with KDC is K_A , K_B

KDC mediated Authentication

- The authentication proceeds as given below.



- 1. Alice sends ID_A, ID_B to KDC notifying that she wants to talk with Bob.
- 2. KDC generates session key K_{AB} and sends it to Alice by encrypting it with K_A appending Ticket T for Bob.

KDC mediated Authentication

- If message 2 is not replay then it authenticates both Alice and Bob.
- 3. Alice sends ID_A and T to Bob notifying that she is Alice and wants to talk with Bob.
- In this way, session key can be shared.
- However, this method is not secure.
- Suppose, the key of Bob is compromised by Turdy
- Also suppose that Turdy has recorded message 2.
- Now, Bob changes its key to K'_B
- Turdy can still use old key K_B .

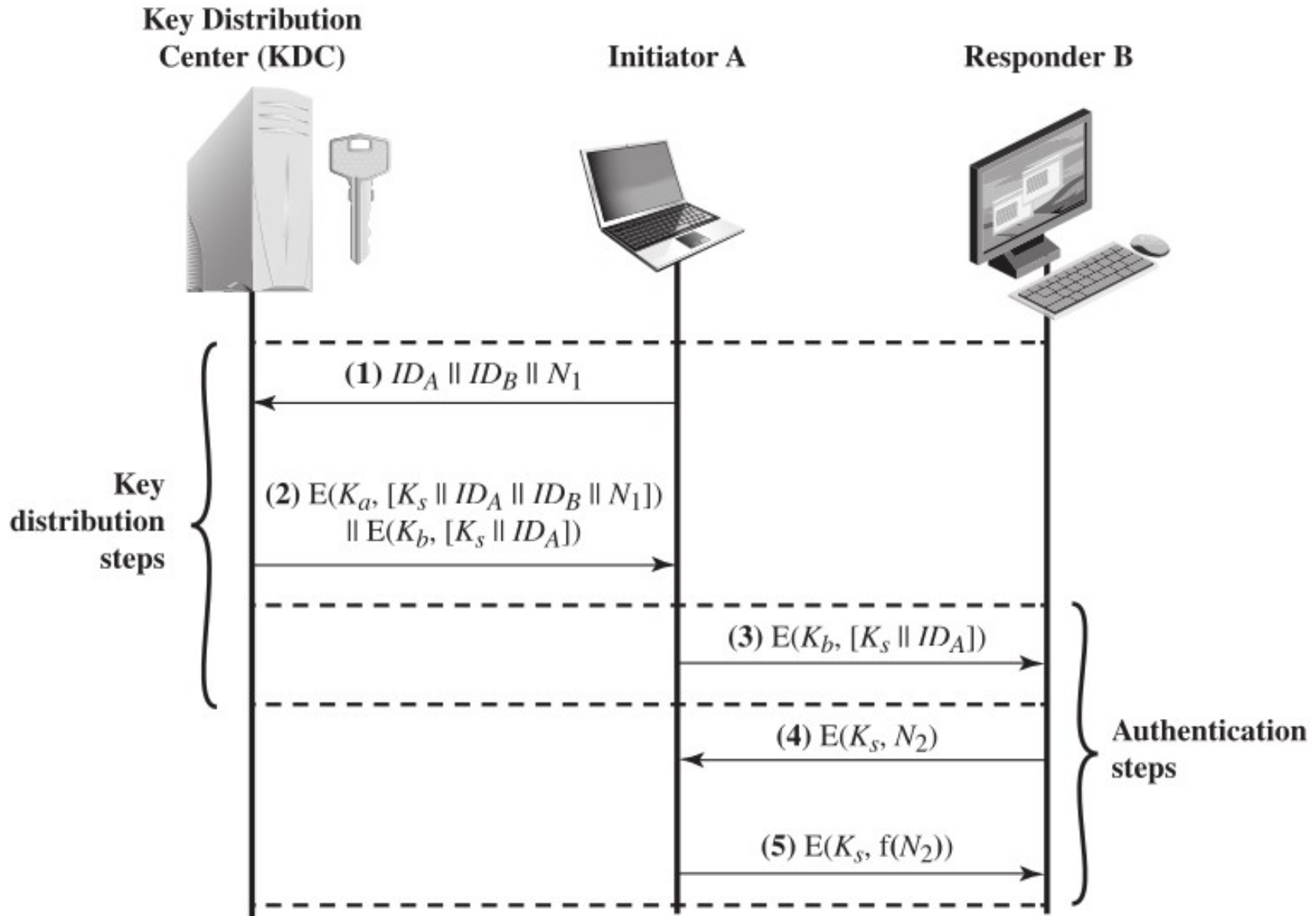
KDC mediated Authentication

- When Alice again contacts to KDC to communicate with Bob, then Turdy impersonates as KDC to Alice.
- Turdy generates new session key, creates new ticket encrypted with Bob's old key K_B .
- When Alice sends this ticket to Bob, Turdy now impersonates as Bob to Alice.

Needham-Schroeder Scheme

- The better Authentication approach is Needham-Schroeder protocol.
- It uses two level symmetric keys to provide confidentiality.
- This approach is quite common.
- Also it is relevant to an understanding of the Kerberos mechanisms.
- The authentication goes as follows.

Needham-Schroeder Scheme



Needham-Schroeder Scheme

- K_A and K_B are secret key of Alice and Bob shared with the KDC respectively.
- 1. Alice sends ID_A and ID_B and nonce N_1 to KDC
- Nonce N_1 prevents Turdy to use Bob's Old key.
- 2. KDC generates session key K_s and sends it to Alice along with Ticket $T = E\{K_B, (K_s || ID_A)\}$ for Bob.
- 3. Alice decrypts the session key and sends the ticket to Bob.
- Then mutual authentication takes place.
- 4. Bob decrypts T and challenges Alice with Nonce N_2 .

Needham-Schroeder Scheme Issues

- Despite the handshake of steps 4 and 5, the protocol is still vulnerable to a form of replay attack.
- Suppose that Turdy has recorded message 2 and has been able to compromise key of Alice.
- Even if Alice changes key, the old ticket to Bob remains valid.
- Turdy impersonate as Alice to Bob and trick Bob into using the old session key by simply replaying old ticket.
- We want to prevent this.

Denning's Improvement

- Denning proposes to overcome this weakness by a modification to the Needham/Schroeder protocol that includes the addition of a timestamp to steps 2 and 3.
- Her proposal assumes that the master keys, K_a and K_b , are secure, and it consists of the following steps.

1. $A \rightarrow KDC: ID_A \parallel ID_B$
2. $KDC \rightarrow A: E(K_a, [K_s \parallel ID_B \parallel T \parallel E(K_b, [K_s \parallel ID_A \parallel T])])$
3. $A \rightarrow B: E(K_b, [K_s \parallel ID_A \parallel T])$
4. $B \rightarrow A: E(K_s, N_1)$
5. $A \rightarrow B: E(K_s, f(N_1))$

Denning's Improvement

- T is a timestamp that assures A and B that the session key has only just been generated.
- Thus, both A and B know that the key distribution is a fresh exchange.
- This needs A and B synchronized with KDC

KERBEROS

- Kerberos allows two users (or client and server) to authenticate each other over an insecure network
- It works under the principal that user's password must never transmit over insecure network.
- Password must's not be stored in client's computer.
- Password must not be stored in unencrypted form in the authentication server database.
- Single sign in per session.

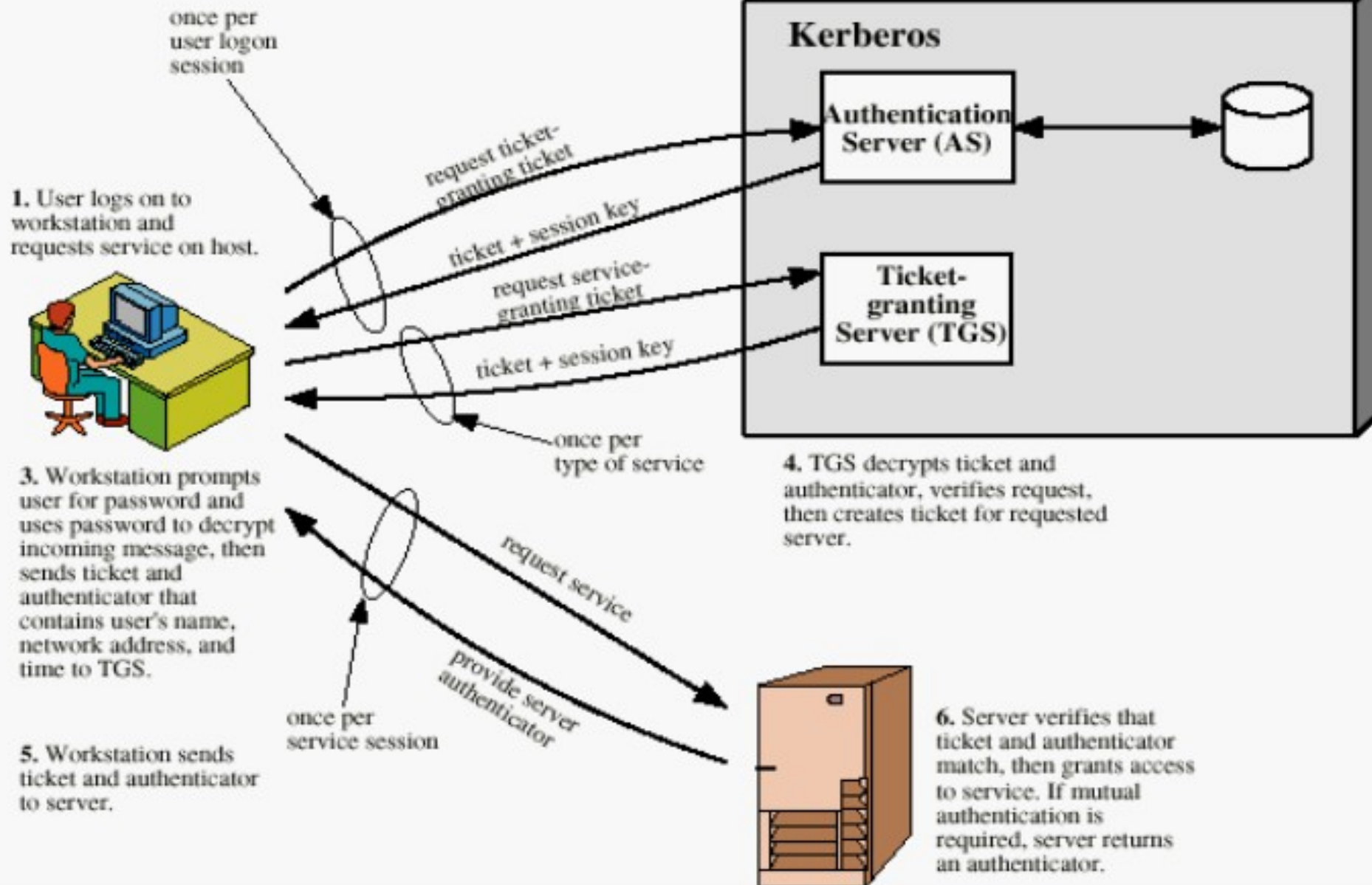
KERBEROS

- Protects against replay attacks
- Uses a trusted third party (Authentication Server) and symmetric key cryptography
- First 3 versions are no longer in use.
- V5 is a generalization of V4 with several problems fixed and additional features.

KERBEROS

- Designed originally for Project Athena at M.I.T.
- Implementation freely available from M.I.T.
- V5 is an Internet Standard (RFC 4120)
- Windows 2000/XP/Server 2003/Vista use Kerberos as their default authentication mechanism
- Apple's Mac OS X clients and servers also use Kerberos
- Apache HTTP Server, Eudora, NFS, OpenSSH, rcp (remote copy), rsh, X window system allow using Kerberos

2. AS verifies user's access right in database, creates ticket-granting ticket and session key. Results are encrypted using key derived from user's password.



KERBEROS V4

- This protocol comprises KDC, Server and Client.
- KDC includes Kerberos components such as Authentication server, Ticket granting server, and database of keys.
- The protocol proceeds as given below

KERBEROS V4

- (1) $C \rightarrow AS$ $ID_C \parallel ID_{tgs} \parallel TS_1$
(2) $AS \rightarrow C$ $E(K_c, [K_{c,tgs} \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2 \parallel Ticket_{tgs}])$
 $Ticket_{tgs} = E(K_{tgs}, [K_{c,tgs} \parallel ID_C \parallel AD_C \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2])$

(a) Authentication Service Exchange to obtain ticket-granting ticket

- (3) $C \rightarrow TGS$ $ID_v \parallel Ticket_{tgs} \parallel Authenticator_c$
(4) $TGS \rightarrow C$ $E(K_{c,tgs}, [K_{c,v} \parallel ID_v \parallel TS_4 \parallel Ticket_v])$
 $Ticket_{tgs} = E(K_{tgs}, [K_{c,tgs} \parallel ID_C \parallel AD_C \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2])$
 $Ticket_v = E(K_v, [K_{c,v} \parallel ID_C \parallel AD_C \parallel ID_v \parallel TS_4 \parallel Lifetime_4])$
 $Authenticator_c = E(K_{c,tgs}, [ID_C \parallel AD_C \parallel TS_3])$

(b) Ticket-Granting Service Exchange to obtain service-granting ticket

- (5) $C \rightarrow V$ $Ticket_v \parallel Authenticator_c$
(6) $V \rightarrow C$ $E(K_{c,v}, [TS_5 + 1])$ (for mutual authentication)
 $Ticket_v = E(K_v, [K_{c,v} \parallel ID_C \parallel AD_C \parallel ID_v \parallel TS_4 \parallel Lifetime_4])$
 $Authenticator_c = E(K_{c,v}, [ID_C \parallel AD_C \parallel TS_5])$

(c) Client/Server Authentication Exchange to obtain service

KERBEROS V4

- Client (C) request for the ticket of Ticket granting server (TGS) with Authentication server (AS) in 1.
- 1. $C \rightarrow AS \quad ID_C || ID_{tgs} || TS_1$
- Where, ID_C = ID of client
- ID_{tgs} = ID of ticket granting server
- TS_1 = time stamp of the message.
- The ID_C helps the AS to get its master key.
- Master keys are used to share the session key.
- ID_{tgs} helps AS to create ticket for specific TGS

KERBEROS V4

- TS_1 helps AS that request is not replay.
- Time stamp consists of current date and time.
- If the request not recent, it will be discarded.
- Clock of C and AS must be synchronized.

KERBEROS V4

- AS challenges C by sending encrypted session key and ticket of TGS with master key of C
- 2. $AS \rightarrow C \quad E(K_c, [K_{c,tgs} || ID_{tgs} || TS_2 || Lifetime_2 || Ticket_{tgs}])$
- Where, K_c = Master key of C with KDC
- $K_{c,tgs}$ = Session key between C and TGS
- TS_2 = Time stamp of message 2
- $Ticket_{tgs}$ = Ticket to be sent to TGS
- $Ticket_{tgs} = E(K_{tgs}, [K_{c,tgs} || ID_c || AD_c || ID_{tgs} || TS_2 || Lifetime_2])$

KERBEROS V4

- AD_C is the network address of C.
- If C can decrypt this message then C and AS are said to authenticated each other.

KERBEROS V4

- C sends $\text{Ticket}_{\text{tgs}}$ to TGS and request for ticket of Server V.
- 3. $C \rightarrow \text{TGS} \quad \text{ID}_v \parallel \text{Ticket}_{\text{tgs}} \parallel \text{Authenticator}_c$
- Where, $\text{ID}_v = \text{ID of } V$
- $\text{Authenticator}_c = E(K_{c,\text{tgs}} [\text{ID}_c \parallel \text{AD}_c \parallel \text{TS}_3])$
- $\text{TS}_3 = \text{Time stamp of message 3}$
- Authenticator_c is encrypted with $K_{c,\text{tgs}}$ which helps to authenticate C with TGS.

KERBEROS V4

- TGS challenges C by sending Ticket of S encrypted with $K_{c, tgs}$
- 4. TGS $\rightarrow$ C $E(K_{c, tgs} [K_{c, v} || ID_v || TS_4 || Ticket_v])$
- Where, $K_{c, v}$ is session key between C and S
- TS_4 = Time stamp of message 4
- $Ticket_v$ = ticket of V
- $Ticket_v = E(K_v [K_{c, v} || Id_c || Ad_c || Id_v || TS_4 || Lifetime_4])$
- K_v = master key of V with AS
- $K_{c, v}$ = Session key between C and V

KERBEROS V4

- C sends Ticket_v and Authenticator_c to V and request for authentication.
- 5. $C \rightarrow V$ $\text{Ticket}_v \parallel \text{Authenticator}_c$
- $\text{Authenticator}_c = E(K_{c,v} [\text{ID}_c \parallel \text{Ad}_c \parallel \text{TS}_5])$
- V decrypts the ticket and obtains the $K_{c,v}$
- Then V sends challenge to C
- $V \rightarrow C$ $E(K_{c,v} [\text{TS}_5 + 1])$
- C decrypts the message and C and V are said to be authenticated each other.

Replicated KDC Kerberos

- A serious problem with having all authentication rely on a single KDC is that it is a single point of failure.
- In such case, it is impossible to access remote resources.
- In addition, a single KDC might be a performance bottleneck.
- For these reasons it is desirable to have multiple KDCs, where each KDC is interchangeable with every other KDC.

Replicated KDC Kerberos

- They share the same master KDC key and have identical databases.
- One site hold the master database to which any updates must be made.
- An update consists of adding an entry for $\langle \text{principal name, key} \rangle$.
- Other sites download the database periodically, either on a timer or as a result of a human issuing a command.

KERBEROS Realm

- Replicated KDCs, poses serious problem of trust.
- It would be hard for several competing organizations such as universities, banks, and government agencies to trust on agency that manages the KDC.
- The agency can access every user's master key, and therefore access everything that every user can access.
- Everyone must also trust the physical controls around every replica of the KDC.

KERBEROS Realm

- The principals in the network are divided into realms.
- Each realm has its own KDC database.
- There can be multiple KDCs in a realm, but they would be equivalent, have the same KDC master key, and have the same database of principals' master keys.
- Two KDCs in different realms, however, would have different KDC master keys and totally different principals' master key databases.

KERBEROS Realm

- In a realm, the Kerberos server must have the user ID and hashed passwords of all participating users in its database.
- All users are registered with the Kerberos server.
- The Kerberos server must share a secret key with each application server.
- All servers are registered with the Kerberos server.

KERBERC

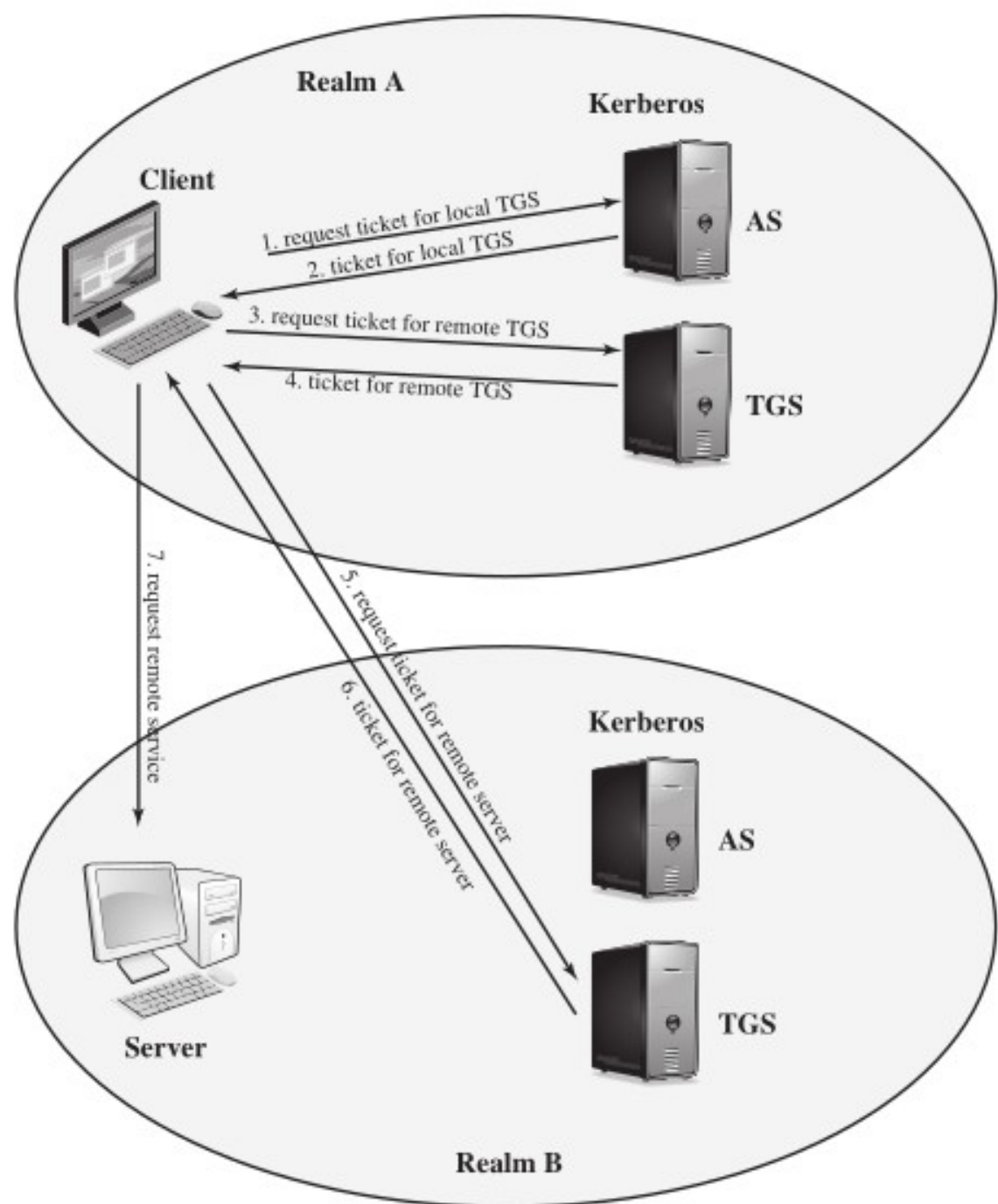


Figure 15.3 Request for Service in Another Realm

KERBEROS Interrealm

- Users in one realm may need access to servers in other realms provided that those users are authenticated.
- Kerberos provides a mechanism for supporting such interrealm authentication.
- For two realms to support interrealm authentication, a third requirement is added:
- The Kerberos servers in interoperating realms share a secret key.
- The two Kerberos servers are registered with each other.

KERBEROS Principal

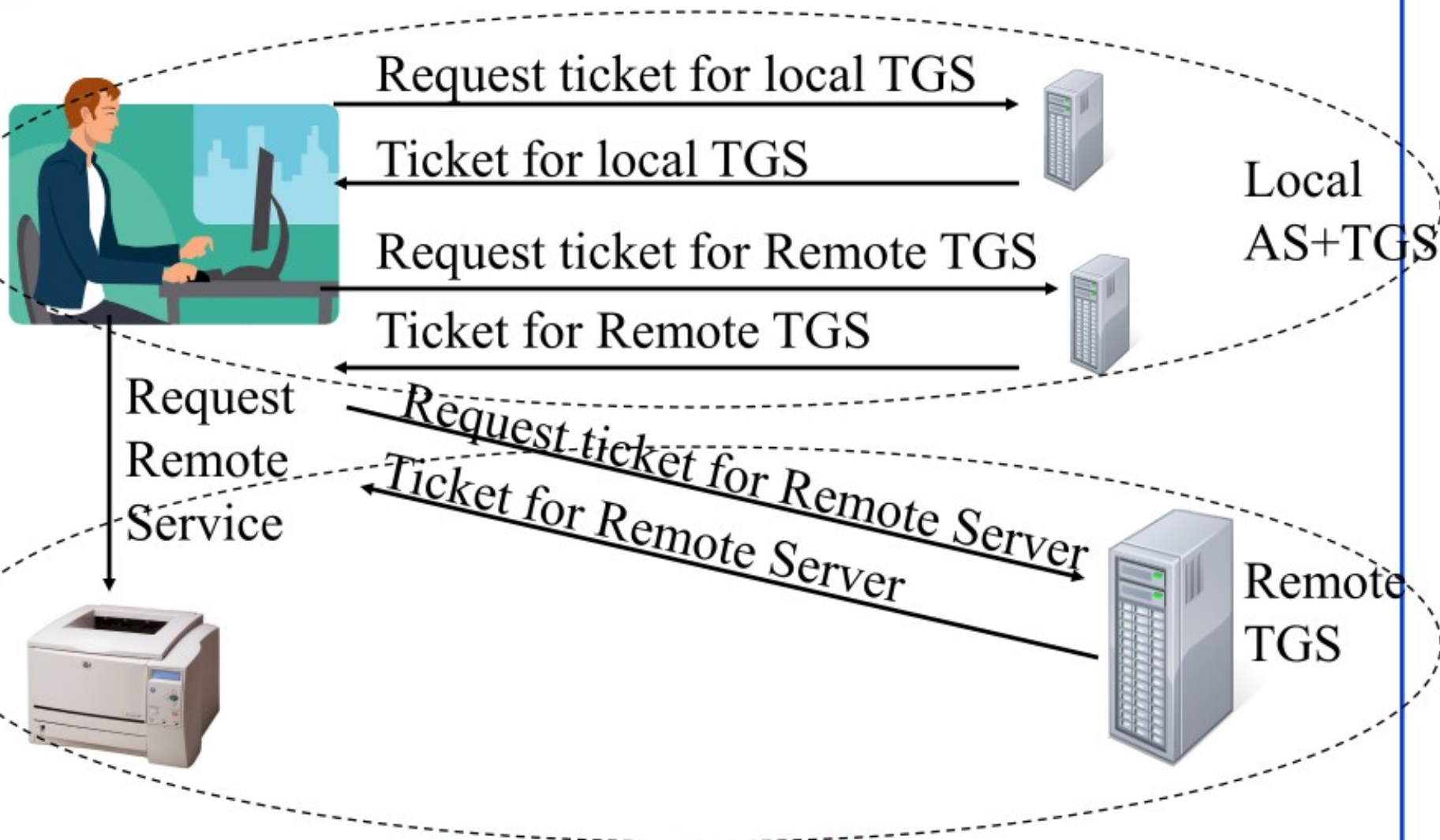
- A client or application server in the Kerberos service is identified by its principal.
- A principal is a unique identity to which the KDC can assign tickets.
- A principal can be a user, such as jdoe, or a service, such as nfs.
- By convention, a principal name is divided into three components: the primary, the instance, and the realm.

KERBEROS Principal

- A typical Kerberos principal would be, for example, `jdoe/admin@CORP.EXAMPLE.COM`.
- In this example: `jdoe` is the primary.
- The primary can be a user name, as shown here,
- It can be service, such as `nfs`. which signifies that is set up to provide various network services, `ftp`, `scp`, `ssh`, and so on.

Inter-Realm Authentication

- Realm: One AS and its clients and servers



KERBEROS V4 Issues

- Names, Instance, Realm (non standard). Limited to 40 Char.
- Only DES encryption. Not strong.
- Only IPv4 addresses. No IPv6 or ISO CLNP addresses.
- Maximum life time limited to 21 hours: 8 bit life time in units of 5 minutes
- No delegation. A server cannot access another server on behalf of the client.
- Inter-realm authentication limited to pairs

KERBEROS V5

- Kerberos v5 was developed to overcome the shortcomings of Kerberos v4.
- Kerberos v5 is superior to Kerberos v4 on following aspects.

KERBEROS V5

- In version 5, ciphertext is tagged with an encryption-type identifier so that any **encryption technique** may be used.
- Version 5 network addresses are tagged with type and length, allowing any **network address type** (IP V6) to be used.
- All message structures are defined using **Abstract Syntax Notation One** (ASN.1) and **Basic Encoding Rules** (BER), which provide an unambiguous byte ordering.

KERBEROS V5

- In version 5, tickets include an explicit start time and end time, allowing tickets with **arbitrary lifetimes**.
- A client issues a request to a print server that then accesses the client's file from a file server, using the client's credentials for access. This **authentication forwarding** is available in v5.
- Version 5 supports a method that requires fewer relationships for interrealm authentication.

KERBEROS V5

- (1) $C \rightarrow AS$ $Options \parallel ID_c \parallel Realm_c \parallel ID_{tgs} \parallel Times \parallel Nonce_1$
(2) $AS \rightarrow C$ $Realm_c \parallel ID_C \parallel Ticket_{tgs} \parallel E(K_c, [K_{c,tgs} \parallel Times \parallel Nonce_1 \parallel Realm_{tgs} \parallel ID_{tgs}])$
 $Ticket_{tgs} = E(K_{tgs}, [Flags \parallel K_{c,tgs} \parallel Realm_c \parallel ID_C \parallel AD_C \parallel Times])$

(a) Authentication Service Exchange to obtain ticket-granting ticket

- (3) $C \rightarrow TGS$ $Options \parallel ID_v \parallel Times \parallel Nonce_2 \parallel Ticket_{tgs} \parallel Authenticator_c$
(4) $TGS \rightarrow C$ $Realm_c \parallel ID_C \parallel Ticket_v \parallel E(K_{c,tgs}, [K_{c,v} \parallel Times \parallel Nonce_2 \parallel Realm_v \parallel ID_v])$
 $Ticket_{tgs} = E(K_{tgs}, [Flags \parallel K_{c,tgs} \parallel Realm_c \parallel ID_C \parallel AD_C \parallel Times])$
 $Ticket_v = E(K_v, [Flags \parallel K_{c,v} \parallel Realm_c \parallel ID_C \parallel AD_C \parallel Times])$
 $Authenticator_c = E(K_{c,tgs}, [ID_C \parallel Realm_c \parallel TS_1])$

(b) Ticket-Granting Service Exchange to obtain service-granting ticket

- (5) $C \rightarrow V$ $Options \parallel Ticket_v \parallel Authenticator_c$
(6) $V \rightarrow C$ $E_{K_{c,v}}[TS_2 \parallel Subkey \parallel Seq \#]$
 $Ticket_v = E(K_v, [Flag \parallel K_{c,v} \parallel Realm_c \parallel ID_C \parallel AD_C \parallel Times])$
 $Authenticator_c = E(K_{c,v}, [ID_C \parallel Relam_c \parallel TS_2 \parallel Subkey \parallel Seq \#])$

(c) Client/Server Authentication Exchange to obtain service

KERBEROS V5

- Message (1) is a C's request to AS for Ticket.
- As before, it includes the ID of the C and TGS
- **Realm_c**: Indicates realm of C
- **Options**: Used to request that certain flags be set in the returned ticket
- **Times**: Used by C to request the following time settings in the ticket:
 - **from**: the desired start time for the requested ticket
 - **till**: the requested expiration time for the requested ticket

KERBEROS V5

- **rtime**: requested renew-till time
- **Nonce₁**: A random value to be repeated in message (2) to assure that the response is fresh and has not been replayed by an opponent

KERBEROS V5

- Message (2) returns a ticket-granting ticket, identifying information for C
- **Ticket**_{tgs} = ticket-granting ticket.
- **K**_{c,tgs} = session key between the C and TGS
- **K**_c = password of C
- The ticket itself includes the session key, identifying information for the client,
- the requested time values, and flags that reflect the status of this ticket and the requested options.

KERBEROS V5

- Message (4) has the same structure as message (2).
- It returns a ticket plus information needed by the client, with the information encrypted using the session key now shared by the client and the TGS.

KERBEROS V5

- Finally, for the client/server authentication exchange, several new features appear in version 5.
- In message (5), the client may request as an option that mutual authentication is required.
- The authenticator includes several new fields:
- Subkey: The client's choice for an encryption key to be used to protect this specific application session.
- Sequence number: An optional field that specifies the number to be used by the server for messages sent to the client during this session.

KERBEROS V5

- If mutual authentication is required, the server responds with message (6).
- This message includes the timestamp from the authenticator.
- It is not possible for an opponent to create message (6) without knowledge of the appropriate encryption keys.
- The subkey field, if present, overrides the subkey field, if present, in message (5).

KERBEROS V5: flags

INITIAL	This ticket was issued using the AS protocol and not issued based on a ticket-granting ticket.
PRE-AUTHENT	During initial authentication, the client was authenticated by the KDC before a ticket was issued.
HW-AUTHENT	The protocol employed for initial authentication required the use of hardware expected to be possessed solely by the named client.
RENEWABLE	Tells TGS that this ticket can be used to obtain a replacement ticket that expires at a later date.
MAY-POSTDATE	Tells TGS that a postdated ticket may be issued based on this ticket-granting ticket.
POSTDATED	Indicates that this ticket has been postdated; the end server can check the authtime field to see when the original authentication occurred.
INVALID	This ticket is invalid and must be validated by the KDC before use.
PROXIABLE	Tells TGS that a new service-granting ticket with a different network address may be issued based on the presented ticket.
PROXY	Indicates that this ticket is a proxy.
FORWARDABLE	Tells TGS that a new ticket-granting ticket with a different network address may be issued based on this ticket-granting ticket.
FORWARDED	Indicates that this ticket has either been forwarded or was issued based on authentication involving a forwarded ticket-granting ticket.